Enabling Prefix Sum Parallelism Pattern for Recurrences with Principled Function Reconstruction

Yang Xia
The Ohio State University
Columbus, OH, USA
xia.425@osu.edu

Peng Jiang
The Ohio State University
Columbus, OH, USA
jiang.952@osu.edu

Gagan Agrawal
The Ohio State University
Columbus, OH, USA
agrawal@cse.ohio-state.edu

ABSTRACT

Much research work has been done to parallelize loops with recurrences over the last several decades. Recently, *sampling-and-reconstruction* method was proposed to parallelize a broad class of loops with recurrences in an automated fashion, with a practical runtime approach. Although the parallelized codes achieve linear scalability across multi-cores architectures, the sequential merge inherent to this method makes it not scalable on many-core architectures, such as GPUs. At the same time, existing parallel merge approaches used for simple reduction loops cannot be directly and correctly applied to this method.

Based on this observation, we propose new methods to merge partial results in parallel on GPUs and achieve linear scalability. Our approach involves refined runtime-checking rules to avoid unnecessary runtime check failures and reduce the overhead of reprocessing. We also propose *sample converge* technique to reduce the number of sample points so that communication and computation overhead is reduced. Finally, based on GPU architectural features, we develop optimization techniques to further improve performance.

Our evaluation results of a set of representative algorithms show that our parallel merge implementation is substantially more efficient than sequential merge, and achieves linear scalability on different GPUs.

CCS CONCEPTS

• **Theory of computation → Massively parallel algorithms**;

KEYWORDS

prefix-sum, sample-and-reconstruction, GPU optimizations, parallel merge, runtime check, communication

ACM Reference Format:
Yang Xia, Peng Jiang, and Gagan Agrawal. 2019. Enabling Prefix Sum Parallelism Pattern for Recurrences with Principled Function Reconstruction. In *Proceedings of the 28th International Conference on Compiler Construction (CC '19), February 16–17, 2019, Washington, DC, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3302516.3307354

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CC '19, February 16–17, 2019, Washington, DC, USA
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-6277-1/19/02.
https://doi.org/10.1145/3302516.3307354

1 INTRODUCTION

Prefix sum is a well known computation pattern that can be parallelized despite dependences. Given an array $[x_1, x_2, \ldots, x_n]$ of n elements, the prefix sum problem is to compute the values of $S_i = x_1 \oplus x_2 \oplus \ldots \oplus x_i$ for $i = 1, \ldots, n$, where $\oplus$ is an associative operator. Due to the associativity, any adjacent elements (e.g., x_j and x_{j+1}) can be merged without waiting for the value of S_{j-1}, and the value of S_{j+1} can be computed as $S_{j-1} \oplus (x_j \oplus x_{j+1})$ once the value of S_{j-1} is available. This allows us to compute the values of S_i's as a hierarchical merge where, half of the elements are reduced in each stage and there are $O(log(n))$ stages in total. If the computation in each stage is performed in parallel, it means that we need execution time $T = O(log(n))$ to compute all the prefix sums[1].

Based on ideas around parallel prefix sum, many efforts have been made over the last decade to parallelize a class of *seemingly sequential* loops that involve recurrences [3, 7–9, 21]. These loops are hard to parallelize because there are strong dependencies across the iterations. An example of such loops is the evaluation of a polynomial expression ($y = \sum_{i=0}^{N}(a[N-i] \cdot c^i)$) by Horner's method:

```
y = 0; for(i=0; i<n; i++) {y = c * y + a[i];}
```

The loop seems hard to parallelize at first glance because there is a loop-carried dependence on y across iterations. However, if we consider the computation across iterations as an associative composition of the loop body functions, we get

$$\begin{aligned} & f(f(y_{i-1}, a[i]), a[i+1]) \\ =& \left(f'(a[i+1]) \oplus f'(a[i])\right)(y_{i-1}) \\ =& c * (c * y_{i-1} + a[i]) + a[i+1] \\ =& \boxed{c * c} * y_{i-1} + \boxed{c * a[i] + a[i+1]} \end{aligned} \tag{1}$$

where f' is a function of c and $a[i]$ and $\oplus$ is an associative operator for composing f'. Thus, the loop can be parallelized as a prefix sum [13].

The most important step in the above parallelization is to identify f' in the loop body and derive a closed-form composition as boxed in (1). This is not an easy task, especially when a loop body contains conditional branches. Most of the previous works rely on a heuristic search and symbolic reasoning to perform the task [3, 7–9, 19]. A downside of these search-based approaches is that they incur a large compile-time or runtime overhead and sometimes fail to find a solution because the search space is exponential.

Recently, Jiang et al.[12] proposed a *sampling and reconstruction* method to identify prefix sum and reduction patterns in recurrences. They observe that the loop body functions in many recurrences are simply linear, rectified-linear, or finite-state functions. Instead of performing symbolic search of closed-form compositions, they

determine if a loop can be parallelized by verifying the type of the loop body function. Once the type of the function is verified, they can start execution from any iteration by using one or multiple selected sampling initial values without knowing the actual values of the recurrence variables. The merge of different chunks are performed by giving the sample points and the actual initial values to a generalized *interpolation* procedure [12].

We now explain this idea taking the *polynomial evaluation* loop above as an example. We can start execution from any iteration i by computing on two values of y at the end of the iteration $i-1$ (y_{i-1}: $y_{i-1,0} = 0$ and $y_{i-1,1} = 1$. This will give us two resulting values of y at the end of iteration $j \geq i$: $y_{j,0}$ and $y_{j,1}$. Since y_j is a linear function of y_{i-1} and we already know two sample points on the line of the linear function, we can compute the actual value of y_j directly when the actual value of y_{i-1} is available. This *sampling-and-reconstruction* strategy works for many other loops with more complex dependences including cases that cannot be parallelized by a state-of-the-art implementation of the search-based methods. Readers are referred to [12] for more details.

Jiang *et al.* also show empirically that their method achieves linear scalability across multiple cores on CPUs [12]. However, a limitation of the sampling-and-reconstruction approach is that it only supports sequential propagation of the actual recurrence values. This is because it requires the actual value to be in the range of the sampling intial values to ensure the correctness of *interpolation*. If the actual value of a recurrence variable is out of range, they need to re-execute the chunk of loop iterations with the actual value. (More detailed illustration of this problem is in Section 3.1.) This becomes a severe bottleneck in achieving scalability when we have a large number of cores and want to merge the values of different chunks hierarchically (for example, as in the parallel prefix sum). In fact, Jiang *et al.* use a sequential merge procedure to propagate the actual values of recurrence variables from the first chunk to the last. They can achieve execution time $T_{par} = O(n/p)$ with $p \ll n$ processors. Although their parallelization method is *cost-optimal* (i.e., $T_{par} \times p = O(T_{seq})$), it cannot achieve $O(log(n))$ execution time with $O(n)$ processors.

Cost-optimal parallelization might be enough in the multi-core era when the number of processors is usually much smaller than the problem size. However, the rise of GPUs and other throughput-oriented processors in recent years provides us massive parallelism in hardware. For such systems, scalability can become a limitation of many parallel implementations that are efficient on multi-core systems.

In this work, we address the scalability limitation of the sampling-and-reconstruction method for parallelizing recurrences on GPUs. To enable prefix sum parallelism pattern for these loops, we propose a set of runtime-checking rules to avoid unnecessary rollbacks. In addition, if the runtime check really fails, we propose an algorithm to reduce the overhead of reprocessing. Further, we propose *sample converge* technique to reduce the number of sample points for a large class of algorithms that satisfy a certain condition. As a result, communication and computation overhead of these algorithms are reduced. Finally, we also adopt effective optimizations such as data transformation and persistent-thread-model to further improve performance on GPUs. We evaluate our methods on a set of representative algorithms. The results demonstrate that our runtime-checking rules are effective and with a low-overhead. In addition, our optimization techniques are effective to improve performance. As a general result, our parallel merge implementation achieves linear scalability on both Nvidia Tesla P100 and Nvidia Tesla K80, and it is several times more efficient than sequential merge implementation.

2 BACKGROUND AND MOTIVATION

In this section, we first introduce background on GPU architectures and sampling-and-reconstruction strategy. Then, we illustrate the motivation of our work using an experiment.

2.1 GPU Architectures

From the viewpoint of software, there are three levels of parallelism on GPUs. [1] The first level is called *warps*, which are consecutive 32 threads that execute instructions in a lock-step. Threads within a warp can exchange register variables using efficient shuffle instructions. The second level is *thread blocks*. Each thread block can hold up to 1024 threads and is assigned to one of the GPU's multiprocessor units, which are called SMs. Threads within the same thread block are allowed to access shared memory, which is much more efficient than accessing the global memory of the GPU, but still slower than shuffle instructions that threads within a warp can use. Besides, CUDA provides efficient instructions to implement barriers among all threads within a thread block. Finally, the third level of parallelism is the *grid*. Threads from different thread blocks within a grid can only communicate through global memory, which is relatively slow. If all threads within a warp simultaneously access global memory locations that lie in the same aligned 128-byte segment, the hardware *coalesces* them into one memory transaction. Otherwise, threads within a warp access multiple 128-byte segments, and the global memory bandwidth is not fully utilized. In addition, there is no grid-wide barrier among threads of the same grid, so programmers need to implement global barriers by themselves [10, 23].

2.2 Sampling-and-Reconstruction Strategy

Recently, Jiang et al. [12] proposed a new *sampling-and-reconstruction* method to parallelize loops with recurrences. We have described their main idea with a linear function example in the Introduction section. We now briefly summarize how to apply their techniques on *rectified-linear* functions and *multivariate* funcitons. Rectified-linear functions arise in loops that have a recurrence variable in conditionals. Figure 1 shows an example of rectified-linear function, which is to compute the maximum segmented sum of an array. In this loop, s is a rectified-linear function of itself after each iteration. The shape of the function is shown as Figure 2(a). A key observation is that the composition of any two rectified-linear functions with the shape Figure 2(a) still holds the same shape – more specifically, the resulting value of s after an arbitrary number of iterations is a rectified-linear function of its initial value and the shape of this rectified-linear function is still Figure 2(a).

For rectified-linear functions, it turns out that four sample points are sufficient to reconstruct the actual function: Let the four sample points be $(x_i[0], y_i[0])$, $(x_i[1], y_i[1])$, $(x_i[2], y_i[2])$ and $(x_i[3], y_i[3])$

[1] In this paper, we use terms specific to the CUDA programming model.

```
m = INT_MIN; s = 0;
for( i = 0; i < N; i++){
   if(s + A[i] < 0) s = 0; else s = s + A[i];
   if(m < s) m = s;
}
```

Figure 1: mss loop: computes maximum segmented sum of an array.

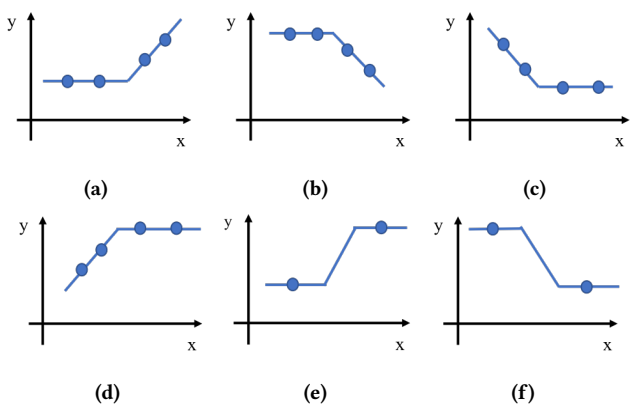


Figure 2: Rectified-linear functions and their combinations.

where $x_i[0] < x_i[1] < x_i[2] < x_i[3]$. Now, as long as ($x_i[0]$, $y_i[0]$), ($x_i[1]$, $y_i[1]$) reside on the left piece of the function and ($x_i[2]$, $y_i[2]$), ($x_i[3]$, $y_i[3]$) fall on the right piece, we can compute any value of the function because we can reconstruct the function through these sample points. More specifically, if the function has one of the shapes among Figure 2(a) to Figure 2(d), we can compute any value in the function by:

$$f(x) = \begin{cases} max(l, r) & : y_i[0] > y_i[1] || y_i[2] < y_i[3] \\ min(l, r) & : otherwise \end{cases}$$

If xi falls on the left piece of the function, l is computed as:

$$l = (y_i[1] - y_i[0]) = (x_i[1] - x_i[0]) * (x_i - x_i[0]) + y_i[0]$$

Similarly, if x_i falls on the right piece of the function, r can be computed as:

$$r = (y_i[3] - y_i[2]) = (x_i[3] - x_i[2]) * (x_i - x_i[2]) + y_i[2]$$

In addition, when the function passes the origin and/or has slope of 1 or -1, fewer sample points are required to reconstruct the function. For example, if the function has shape Figure 2(a) and the slope of function is 1 on the right piece, then two sample points are sufficient : $x_i[0]$ on the left piece and $x_i[3]$ on the right piece, and the value at any point of the function can be computed as $y = max(y_i[0], x_i + y_i[3] - x_i[3])$. The *mss* algorithm above is an example of this kind of function.

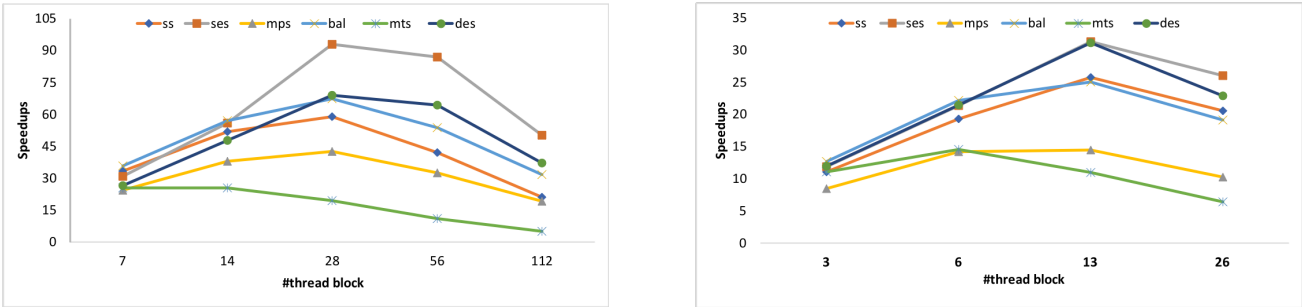
In addition, there is another complication. As illustrated above, the sampling strategy above requires that ($x_i[0]$, $y_i[0]$), ($x_i[1]$, $y_i[1]$) fall on the left piece and ($x_i[2]$, $y_i[2]$), ($x_i[3]$, $y_i[3]$) fall on the right piece of the function. However, since the turning point of the function is unknown at the compilation time, we need to do a runtime check to guarantee correctness. Specifically, we check whether the actual initial value x is between $x_i[1]$ and $x_i[2]$, which are initial values of two middle sample points. If it is the case, true results can be derived through the reconstruction function above. Otherwise, we need to *rollback* and re-execute the set of iterations with the actual initial value.

Finally, it is common in real loops that a recurrence variable depends on multiple recurrence variables. Take the *mss* loop in Figure 1 as an example, the value of m at the end of iteration i (m_i) is a rectified-linear function of its value at the end of iteration $i - 1$ (m_{i-1}) and the value of s at the end of iteration $i - 1$ (s_{i-1}). Therefore, m_i is a bivariate function of both s_{i-1} and m_{i-1}. For a multivariate function, it can be decomposed into several dimensions. As long as the function type along each dimension is unique and can be parallelized using sampling-and-reconstruction method, a multivariate function can be reconstructed through *interpolation*. Here, we give an example to briefly illustrate the procedure of interpolation. Consider a function $z = f(x, y) = a * x + b * y + c$, $f(0, 1) = z1$, $f(1, 0) = z2$, $f(1, 1) = z3$, the value of z at any point (x, y) can be computed in the following steps: 1) Because function $f(x, 1)$ is linear, $f(x, 1) = (z3 - z1) * x + z1$. 2) Similar to the first step, we can obtain $f(x, 0) = (z2 - z0) * x + z0$. 3) Because $f(x, y)$ is a linear function of y determined by $f(x, 0)$ and $f(x, 1)$, we have the reconstructed function as $f(x, y) = (f(x, 1) - f(x, 0)) * y + f(x, 0)$.

2.3 GPU Specific Needs

As explained earlier, the sampling-and-reconstruction method requires sequential merge, with runtime checks and possibility of *rollback*. Despite these potential overheads, in the previous paper, sampling-and-reconstruction method was shown to achieve linear scalability across multi-core architectures [12]. However, we observed that it is not able to achieve linear scalability on GPUs using sequential merge. To demonstrate this scalability problem, we performed experiments on two GPUs with different architectures: an Nvidia Tesla K80, and an Nvidia Tesla P100. Table 2 shows the major specifications of the GPUs used in our experiments. A key specification in the table is *maximal launch*, which is calculated as the maximal number of thread blocks limited by the number of streaming processes. This parameter is related with the size of thread block. In our experiments, we set the thread block as 1024 threads. Since the maximal number of active warps per thread blocks for these GPUs is 64 and each warp contains 32 consecutive threads, the maximal number of thread blocks is 2 ($64*32/1024$) for each streaming processor. Figure 3 shows the relationship between the number of thread blocks and speedups of GPU implementations over sequential programs for a set of representative algorithms. These algorithms are collected from the previous research work [12, 20, 21]. Some key features of these algorithms are shown in Table 3. As indicated in the figure, for all algorithms, the speedups increase in the beginning. However, as the number of thread blocks continues to increase, the speedups become non-linear and even drop within the maximal launch. Further, as we show experimental results for both GPUs, we believe this scalability problem is not specific to one architecture.

We identify that the bottleneck of the above scalability problem is the sequential propagation and merge. The sequential merge seriously hinders the scalability on GPUs for the following reasons:



(a) Scalability Issue on Tesla P100

(b) Scalability Issue on Tesla K80

Figure 3: Speedups with different number of thread blocks on different GPUs

First, as the number of threads increases, it takes more iterations to merge the results. As a result, it causes longer latency for the overall procedure. Second, since there are explicit data dependencies between chunks, a global synchronization is needed after each thread finishes its local processing. The overhead of global synchronization is significant on GPU architectures. Finally, for sequential merge, the sampled results of each thread needs to be written into global memory and the read-back again. This will cause significant memory access overhead.

3 APPROACH FOR MANY-CORE SYSTEMS

In the context of other types of computations such as simple reductions, various *parallel merge* strategies have been proposed in the past to improve scalability [4]. Thus, it is natural to try and use parallel merge with the sampling-and-reconstruction method to achieve highly parallel execution of recurrences. However, due to certain correctness issues, parallel merge cannot be directly and efficiently applied. In this section, we first illustrate the problem in using a parallel merge while applying the sampling-and-reconstruction method focusing on rectified-linear functions. Then, we propose our refined runtime check conditions to avoid unnecessary runtime check failures. In addition, we propose our method to reduce the overhead of reprocessing when runtime check really fails. Finally, we describe our method to reduce the number of sample points for a set of loops so that communication and computation overheads are reduced.

3.1 Problem Description

The key problem with the parallel merge is that the runtime-checking rules used in sequential propagation can cause many rollbacks, and lead to poor performance. With sequential propagation, each chunk receives actual initial value x from the preceding chunk and checks whether it is between two middle sampling initial values, $x_i[1]$ and $x_i[2]$. If so, runtime check succeeds. Otherwise, we rollback and re-execute the corresponding set of iterations with the correct initial value x. Typically, for each chunk i, we choose sampling initial value $x_i[1]$ to be a small value S and $x_i[2]$ to be a large value L. Therefore, the actual initial value x is unlikely to fall out of the range of (S, L), and the reprocessing is avoided in almost all cases.

However, the runtime check is easy to fail with parallel merge using above checking rules. In parallel merge, each chunk receives sampled results of preceeding chunks rather than actual initial values, and updates the sampled results for a bigger chunk. Abstractly, if thread i and $i+1$ both start computation with sampling initial values S and L, each obtains two outputs ($y_i[1]$ and $y_i[2]$ for thread i, $y_{i+1}[1]$ and $y_{i+1}[2]$ for thread $i+1$. Then, thread $i+1$ receives $y_i[2]$, and checks whether it is in the range of (S, L). Since the computation started with L and not an actual value, it is possible that the result $y_i[2]$ is larger than the initial value L and causes the runtime check to fail.

As a concrete example, let us consider the *maximum segmented sum* loop in Figure 1 with an input array A[4] = {3, 2, -4, -1}. Here, we assume that the processing of array A is assigned to 4 threads, each with one iteration, and we only consider the recurrence variable s, which is a rectified-linear function of itself. Since the shape of the function is shown in Figure 2(a) and the slope is 1, two sample points are sufficient for each chunk i: ($x_i[0]$, $y_i[0]$) on the left piece and ($x_i[1]$, $y_i[1]$) on the right piece. The reconstruction function for s is $y = max(y_i[0], x+y_i[1]-x_i[1])$. We take two sampling initial values, which are -100 and 100, respectively, for each chunk. The procedure of sampling execution is shown in the upper part of Figure 4(a) and Figure 4(b). Figure 4(a) illustrates the method of sequential merge. The whole merge stage consists of four sequential steps. In each step, a chunk receives actual initial value from the preceding chunk and calculates the initial value for the next chunk using the reconstruction function. In this example, the actual initial value is always between the two sampling initial values. Thus, thread 3 successfully commits the final result. However, this runtime-check rule is not feasible with parallel merge. As shown in Figure 4(b), a chunk is not able to get its true initial value within a parallel merge. Instead, thread 1 and 3 receive sampled results from previous chunks in parallel in the first step of merge. For thread 3, runtime check succeeds since sampled results of thread 2 are in the range of $(-100, 100)$. As a result, it gets the sampled results for union of processing chunks 2 and 3. Unfortunately, the second sampled result of chunk 1 is 103, which is out of the range. This will cause a runtime check failure and thread 1 will rollback and re-execute chunk 1 with the true initial value. It is important to note that the correctness problem is even not related with the selection of sampling initial values in this example. Assume that the larger sampling initial value is L, the condition that the runtime check

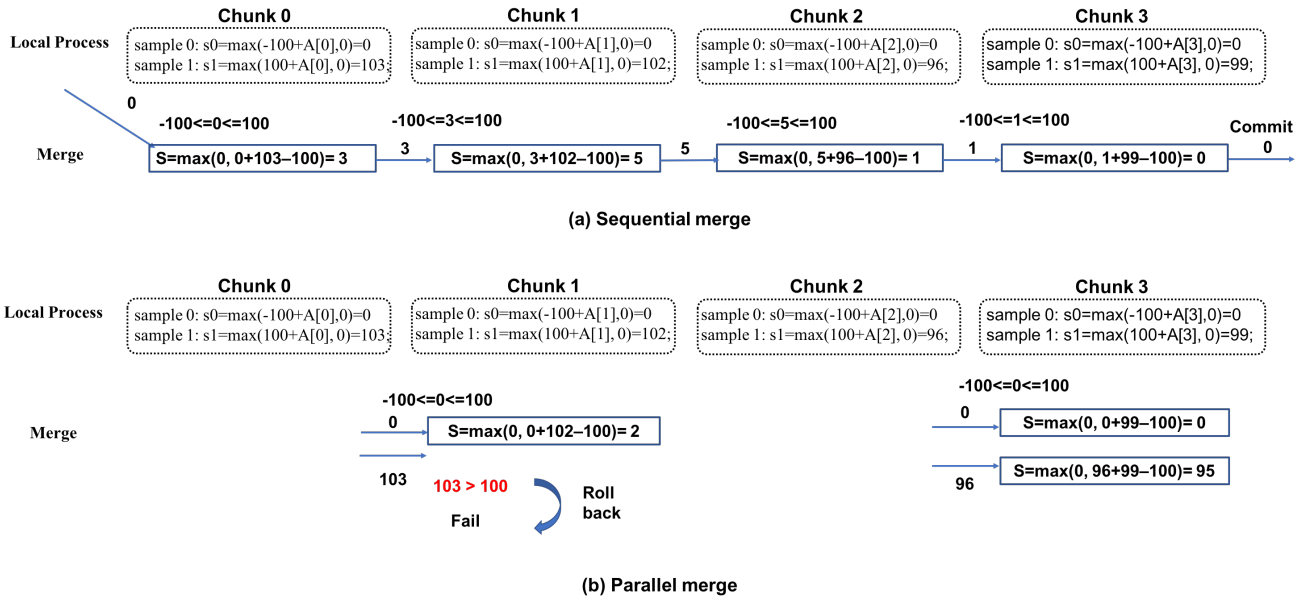


Figure 4: An illustration of runtime-checking problem in parallel merge with mss example.

Table 1: Number of runtime-checking failures in the first step of parallel merge using original runtime-checking rules

Algorithms	#runtime check failures
mss	5833
mps	20886
mts	3818

succeeds for this input data set is $max(0, L + 3) <= L$, which is impossible.

To demonstrate this problem, we performed experiments with some realistic loops on an Nvidia Tesla P100. In this set of experiments, we set the number of thread blocks to be 56, which is equal to the number of streaming processors for this GPU. Within each loop, the input data set contains 128 million data items. The second column of Table 1 shows the number of runtime-checking failures in the first step of parallel merge with original runtime-checking rules. Clearly, there are a significant number of failures for each loop. As a result, the frequent rollback and re-execution makes it take even longer execution time than sequential merge.

3.2 Refined Runtime-Checking Rules for Parallel Merge

A naive solution to solve the above problem is to increase the number of sample points. However, this method will increase the overhead significantly. After a deeper analysis on the purpose of runtime check, we found that the runtime reprocessing within parallel merge implementations can be avoided in most cases. The key point is that although we do not know the actual turning point of the function, we are able to know on which side the sample points are through their values. Then, by comparing the values of sample points with the intial values (actual initial values or sampled results received from previous chunks), we might be able to know the positions of initial values and avoid unnecessary runtime reprocessing.

In the most general case, four sample points are necessary to reconstruct a rectified-linear function. Take a function with the shape Figure2(a) as an example, the distributions of four sample points can be classified into five cases, which are illustrated in Figure 5. Since the function is monotonically increasing, we are able to know which side the sample points reside by comparing values of the four sampled results. Thus, we propose to identify the exact distribution of the four sample points at runtime, and reconstruct the function for each case. In the most ideal case, the distribution of sample points is as *distribution 3* or *distribution 4*, the sampled results can be updated through original reconstruction functions. For *distribution 1* and *distribution 2*, if the actual initial value x is on the left side, we can still get updated sampled results. Otherwise, we need to rollback and reprocess the corresponding iterations. Similary, for *distribution 5*, we can get correct sampled results as long as the actual initial value x is on the right side. Algorithm 1 shows the detailed procedure of our runtime checks.

More efficiently, if the slope of the rectfied-linear function is 1 or -1 or the function passes the origin, the runtime-checking rules can be simplified. For example, if we know the function has shape Figure 2(a) and the function has slope 1 on the right piece, two sample points $(x_i[0], y_i[0])$, $(x_i[1], y_i[1])$ are sufficient for sample-and-construction method. There are three different distributions in this case. In the most ideal case, one sample point is on the left side and the other sample point is on the right side. Then, the sampled results can be updated through original reconstruction functions. Otherwise, the two sample points are both on the left side or right side, and we can still avoid reprocessing as long as we are able to ensure that the initial value x is on the same side. Further, if

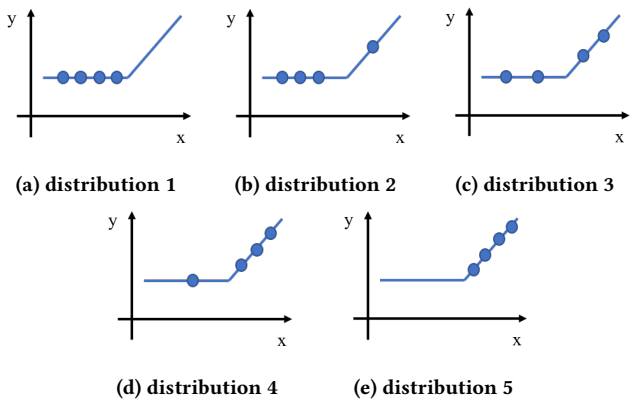


Figure 5: Five different distributions of four sample points for a function with shape Figure 2(a).

Algorithm 1 Refined Runtime-checking Rules for a Rectified-linear Function with Shape in Figure 2(a)

```
if y_i[0] == y_i[1] == y_i[2] == y_i[3] then                    ▷ distribution 1
    if x < x_i[3] then
        y = y_i[0];
    else
        Reprocess;                                              ▷ reprocess case 1
    end if
else if y_i[0] == y_i[1] == y_i[2] then                         ▷ distribution 2
    if x < x_i[2] then
        y = y_i[0];
    else
        Reprocess;                                              ▷ reprocess case 2
    end if
else if y_i[0] == y_i[1] then                                   ▷ distribution 3
    y = max(y_i[0], (y_i[3] − y_i[2])/(x_i[3] − x_i[2]) ∗ x+
        (x_i[3] ∗ y_i[2] − x_i[2] ∗ y_i[3])/(x_i[3] − x_i[2]));
else if (y_i[1]−y_i[0])/(x_i[1]−x_i[0]) == (y_i[3]−y_i[2])/(x_i[3]−x_i[2])
then                                                            ▷ distribution 5
    if x > x_i[0] then
        y = (y_i[1] − y_i[0])/(x_i[1] − x_i[0]) ∗ x+
            (x_i[1] ∗ y_i[0] − y_i[1] ∗ x_i[0])/(x_i[1] − x_i[0]);
    else
        Reprocess;                                              ▷ reprocess case 3
    end if
else                                                            ▷ distribution 4
    y = max(y_i[0], (y_i[3] − y_i[2])/(x_i[3] − x_i[2]) ∗ x+
        (x_i[3] ∗ y_i[2] − x_i[2] ∗ y_i[3])/(x_i[3] − x_i[2]));
end if
```

the slope of the function is known to be 1 or -1, and it passes the origin, only one sample point is needed to reconstruct the function. There are only two different distributions. Take a function with the shape Figure 2(a) as an example, only one sample point ($x_i[0]$, $y_i[0]$) on the left side is needed and the value of the function can be computed as $y = max(y_i[0], x)$. There are only two different distributions now: If $y_i[0] == x_i[0]$, the sampe point is on the right side and we need to roll back and reprocess the chunk. Otherwise, the sample point is on the left side and we can reconstruct the function through our normal method.

3.3 Utilizing Monotonicity of Turning Points

Although the above runtime-checking rules can avoid faulires in most cases, there are still several cases when reprocessing is necessary, which is also indicated in Algorithm 1. Thus, we further refine runtime-checking rules and avoid runtime-checking failures based on the following property.

Monotonicity of Turning Points: *Let $f_1, f_2, \ldots, f_n$ be functions with the shape in Figure 2(a), and let F_i be a function composed functions of i $f_1, f_2, \ldots, f_i$. According to previous knowledge, F_i is still with the shape of Figure 2(a). For the n such resulting functions $F_1, F_2, \ldots, F_n$, let the corresponding turning points of these functions be $FX_1, FX_2, \ldots, FX_n$. Then, the following property holds: $FX_1 <= FX_2 <= \ldots <= FX_n$.*

Proof. Without losing generality, we consider two rectified-linear functions f_1 and f_2 with shape Figure 2(a) as follows:

$$f_1(x) = \begin{cases} C_2 & : x < C_1 \\ A_1 x + C_2 - A_1 C_1 & : x \geq C_1 \end{cases}$$

and

$$f_2(x) = \begin{cases} C_4 & : x < C_3 \\ A_2 x + C_4 - A_2 C_3 & : x \geq C_3 \end{cases}$$

The turning points of f_1 and f_2 are C_1 and C_3, respectively. Let F be the composed function of f_1 and f_2, then, it is also a rectified-linear function with the shape in Figure 2(a). To determine the turning point of F, we consider two cases. Case 1: If $C_3 < C_2$, then the turning point of F is same as function f_1. Case 2: If $C_3 >= C_2$, the turning point of F is $C_1 + (C_3 - C_2)/A_1$. Therefore, in any case, the turning point of F is not less than C_1. Based on this, when more functions with shape in Figure 2(a) are composed, the turning point will not decrease. □

Note that a similar result can be obtained for rectified-linear functions with shape in Figure 2(b)-Figure 2(d) . Utilizing these properties, we are able to obtain more information about the position of the turning point. Consider the *reprocess case 1* in Algorithm 1, where we do not know whether the initial value x is larger than the turning point X or not, we can compute few iterations with both x and the smaller sampling initial value $x_i[3]$ as input, and get results Y_0 and Y_1, respectively. Assume that the function corresponding to these iterations is f_1. If Y_0 is equal to Y_1, then x is smaller than the turning point of function f_1. According to the monotonicity property, x is smaller than the true turning point X. In this case, we avoid rolling back and reprocessing the chunk. Similarly, we can also improve runtime check conditions for *reprocess case 2* and *reprocess case 3* when this monotonicity property is applied.

3.4 Reduce Overhead of Reprocessing

With a naive implementation of parallel merge, the overhead of rollback and reprocessing is significant. Consider using n steps to merge 2^n chunks, if runtime check fails in the last step, half of the entire dataset needs to be re-executed. However, such a huge overhead is unnecessary because we should utilize existing partial merge results. In this subsection, we show how we reduce the reprocessing overhead.

Algorithm 2 Procedure to Merge Sampled Results of Chunk *Left* and Chunk *Current*

```
1:  function MERGE(left, current) tmp = current;
2:      check_result = runtime_check(tmp);
3:      ▷ Step 1: Find a chunk on which runtime check succeeds until reaching
        the first level.
4:      while tmp.level !=0 && check_result == FAILURE do
5:          tmp = tmp.left_child;
6:          check_result = runtime_check(tmp);
7:      end while
8:      ▷ Reaches the first level.
9:      if check_result == FAILURE then
10:         Reprocess chunk tmp;
11:     end if
12:     ▷ Step 2: Merge partial results of chunks until reaching chunk current.
13:     right = calculate_right_sibling(tmp);
14:     Merge(tmp, right);
15:     parent = tmp.parent;
16:     while parent != current.parent do
17:         right_sibling = calculate_right_sibling(parent);
18:         Merge(parent, right_sibling);
19:         parent = tmp.parent;
20:     end while
21: end function
```

Before we describe this algorithm in detail, we briefly illustrate key terms used in our algorithm. In this paper, if chunk C is union of chunk A and chunk B, we call chunk C the *parent* of both chunk A and chunk B. Assume that chunk A precedes chunk B, then chunk A is the *left child* of chunk C and chunk B is the *right child* of chunk C. Algorithm 2 shows the abstract procedure to merge sampled results of two chunks, *left* and *current*. First, it performs runtime check on chunk *current* using above runtime-checking rules. If runtime check succeeds, the sampled results of chunk *left* and *current* can be merged directly. Otherwise, we recursively check *left child* of chunk *current* until the runtime check succeeds or it reaches the first level. In the second step, we first re-execute the chunk of the first level if the runtime check still fails. Then, we continously merge results of the chunk and its right sibling in a hierarchical way until we merge results of the original two chunks successfully.

3.5 Reducing Number of Sample Points

In modern computer systems, especially streaming processors such as GPUs, the typical performance bottleneck is the overhead of data movement, rather than computation. Therefore, it is important to reduce data movement overhead of parallel merge. Although parallel merge needs fewer merge steps (i.e., parallel merge takes $O(log(n))$ steps and sequential merge takes $O(n)$ to finish merge, where n is the number of chunks)), it incurs more data movement overhead than sequential merge.

We start by comparing the communication overhead of these two merge methods when sampling and reconstruction method is applied. With sequential merge, each thread receives its actual initial values from the preceding chunk, and computes the initial values for the next chunk. Thus, the total amount of communication is simply:

$$\#Comm = n_chunk \times n_recurrence$$

where *n_chunk* denotes the number of chunks and *n_recurrence* represents the number of recurrence variables between chunks. As a comparison, for parallel merge, each chunk $i+1$ treats each sampled output of chunk i as an initial value and computes the sampled results for the union of chunk i and chunk $i+1$ through the reconstruction function. Thus, the amount of data communication with parallel merge is :

$$\#Comm = n_chunk \times n_recurrence \times n_sample,$$

where *n_sample* is the number of sample points when we apply sampling-and-reconstruction method for the corresponding algorithm. From the above fomulas, the communication overhead of parallel merge is typically larger than sequential merge as long as *n_sample* is greater than 1. In addition, it is proportional to the number of sample points. Therefore, we propose to seek opportunities to reduce the number of sample points so that data movement ovehead is lowered.

Many previous research works [11, 16, 17, 25, 26] have demonstrated the effectiveness of state-convergence for finite state machine algorithms (FSM). According to their studies, most FSMs, even those with many states, often converge to 16 or less active states for any input. By utilizing this state-convergence property, the computation and communication workload of FSM algorithm is proportional to the number of active states rather than the total number of states.

Based on this observation, we design a sample convergence scheme for our target applications. Specifically, we identify the following conditions: 1) The recurrence variable is assigned with values belonging to a finite set in one branch – a typical example is that the recurrence variable is assigned with a constant variable; 2) The corresponding condition is not related with any recurrence variable. When these two conditions are satisfied, the recurrence variable will convergence once the branch is executed, irrespective of how many sample points there are. We found that a large number of algorithms in realistic applications satisfy these two conditions. Figure 6 shows a concrete example which satisfies the above conditions. It is a user-defined aggregation from Raychev *et al.* [20]. Here, the recurrence variable *count*, which is highlighted in Figure 6, is assigned with a constant variable 0. Second, the condition for that branch statement is $A[i] - A[i-1] >= k$, and this condition is not related with *count*. Based on sampling-and-reconstruction method, this aggregation is an example of linear function and two sample points will be necessary to reconstruct the function. Thus, once *else* branch is executed, these two sample points will converge to same path. As a result, the communication and computation workload is reduced.

4 IMPLEMENTATION DETAILS

In this section, we first briefly illustrate several key performance considerations of implementations on GPUs, then we summarize our overall procedure of parallel merge.

Data Transformation: In a naive implementation, each thread sequentially accesses its own chunk, which is stored in a piece of continuous memory. Since the chunk size is typically large, consecutive threads will access distant memory locations. As a result, memory accesses across threads will not be contiguous and low throughput is achieved from global memory. To overcome this

```
count = 0;
num_result = 0;
for (i = 1; i < N; i++) {
    if (A[i] - A[i - 1] < k) {
        count++;
    } else {
        result[num_result++] = count;
        count = 0;
    }
}
```

Figure 6: A user-defined aggregation that satisfies sample convergence conditions

problem, we change the layout of the input data, with the goal that consecutive threads access consecutive memory locations.

Clearly, such data layout transformation can have overheads. However, in many cases, different algorithms can be executed over the same data set many times, and data set is not updated. For such scenarios, one can justify offline data transformations since the overhead of data transformation is easily amortized.

Persistent-thread Model: On GPUs, programmers can decide how many thread blocks to launch in a grid. However, it is possible that only a subset of these blocks can be executed concurrently on a GPU because of limitations of resources such as the number of registers and the size of shared memory. When a thread block finishes, the hardware scheduler will choose another one to run until the entire grid finishes. This programming model causes several problems, for example, different thread block cannot synchronize and exchange data with each other. To avoid this, we adopt *persistent-thread* model [10]. In persistent-thread model, we only launch as many blocks as can be simultaneously active, and assign multiple work items to each thread.

Selection of Chunk Size: The following trade-off arises while choosing the chunk size on GPUs. The advantages of setting a relatively small chunk size on GPUs is that it achieves better data locality – if each thread only processes several items at one time, the data can reside in registers. This way, we can exploit efficient shuffle instructions to transfer data within a warp. On the other hand, since each thread only processes few data items at one time, it will require several iterations to process the entire data set when input data size is large. At each iteration, there is some overhead to communicate data and perform synchronization among the threads. On the other hand, as we apply data transformation on the input data set, the overhead of coalesced memory access is much reduced even when the chunk size is large. Based on these observations, we make the chunk size as large as possible to reduce the communication and synchronization overhead , which means that the number of chunks is equal to the number of threads.

Based on above implementation points, we implemented a two-level merge on GPUs, which includes a *parallel local-level* merge and a *sequential global level* merge. In the *parallel local-level* merge, each thread computes sampled results of its own chunk and keeps the results in registers. Then, threads within the same warp communicate with each other through efficient shuffle instructions. With 32 threads and parallel merge, it takes 5 iterations before the sampled outputs for the entire warp is calculated and stored in the shared memory. Next, a single warp is used to merge sampled results from different warps and get sampled outputs for the entire thread block. Finally, the results for entire thread block are written back to the shared memory.

In the *sequential global level* merge, we merge sampled results of different thread blocks. In the first step, we write the sampled results of thread blocks to global memory since different thread blocks can only communicate with each other through global memory. Then, a global synchronization is performed to ensure that the sampled results are available for each thread. Finally, we merge the results of different thread blocks in a sequential manner. Since we adopt persistent-thread model, only a fixed number of thread blocks are launched in one grid, and the overhead of the last step is trivial.

Table 2: Specifications of GPUs used in experiments.

GPUs	Tesla K80	Tesla P100
Architecture	Kepler	Pascal
#SM	13	56
maximal launch	26	112
FP32 CUDA Cores/GPU	2496	3584
Base Clock (MHz)	560	1328
Memory Interface	384-bit GDDR5	4096-bit HBM2
Register File Size / SM (KB)	128	256
Shared Memory Size / SM (KB)	112	64

Table 3: Specifications of algorithms used in experiments

Abbreviation	Function Types	#Sampling Sets
mps	linear; rectified-linear	1 * 1
mss	linear; rectified-linear	2 * 1
mts	linear; rectified-linear	2 * 1
ss	linear; rectified-linear	1 * 1
bal	linear; rectified-linear	1 * 1
ses	linear	2
des	linear	2 * 2
agg1	linear; fsm	2 * 2
agg2	linear	2

5 EXPERIMENTAL RESULTS

In this section, we report on a series of experiments that evaluate the effectiveness of our methods using a set of representative algorithms. In the following, we first introduce the setup of our experiments. Then, we demonstrate that the overhead of executing our runtime-checking rules is trivial. We proceed to verify the effectiveness of our sample-convergence technique and data transformation optimization. Finally, we compare the seedups over sequential programs of sequential merge and parallel merge and discuss the difference in scalability between these two merge methods. For generality, we also performed experiments (not shown) on 64 nodes multi-core CPUs and found it achieved 2x performance improvement with parallel merge compared with sequential merge.

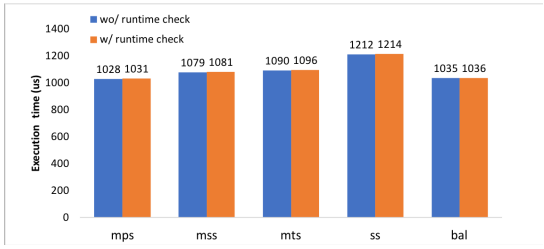


Figure 7: Comparing execution time with and without executing runtime-checking rules.

5.1 Experimental Setup

Hardware and Software Platform: Our experiments were performed on two GPUs with different architectures, an Nvidia Tesla P100 and an Nvidia Tesla K80. Major specifications of these two GPUs are shown in Table 2. The host operating system for our experiments is Red Hat 7.4. We used Clang libtooling to generate cuda kernels automatically. The CUDA programs are based on CUDA 9.2 toolkit and NVCC V9.2.88 is used to compile our programs. All sequential programs are compiled using icpc 18.0.3 with "-O3 -xHost" optimization flag.

Benchmarks: Table 3 shows key features of the algorithms used in our experiments. All of these algorithms are collected from previous research efforts in this area [12, 20, 21]. For other details of these algorithms, readers are referred to [12]. By default, the data size used in our experiments is 128M data items (integers or float point numbers). In reporting execution times, we only consider the computation time on GPU when measuring execution time. The time it takes to transfer the input data from CPU to the GPU or write the results back to the CPUs is not included. We also repeated each experiment three times, and reported the average execution time as the final result. Because the variance in execution times was very small, the range is not reported in our figures.

5.2 Overhead of Executing Runtime-Checking Rules

A major concern with our new runtime-checking rules is that they involve many branch divergences and additional computations, which might hinder performance. However, since many algorithms are memory-bound on GPU architectures, additional computations will not add to the total execution time. To verify this, we compared execution time of our implementations with the ones without executing runtime-checking rules. Figure 7 shows the experimental results on an Nvidia Tesla P100. As indicated in the figure, the overhead of executing our runtime-checking rules is negligible, which is less than 0.1%. Our experiments on an Nvidia Tesla K80 have a similar result.

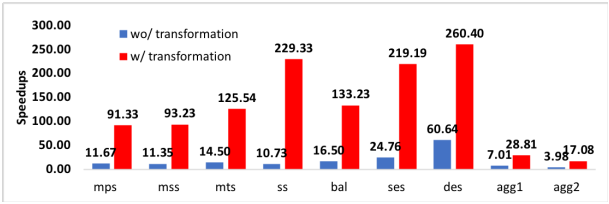


Figure 8: The effectiveness of data transformation for different algorithms on an Tesla P100.

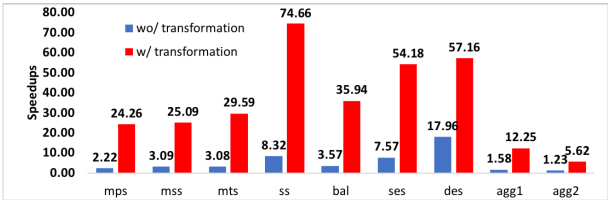


Figure 9: The effectiveness of data transformation for different algorithms on an Tesla K80.

5.3 Benefits from Optimizations

Table 4: Speedups when sample converge is enabled on an Tesla P100 and an Tesla K80.

Algorithms	Speedups on Tesla P100	Speedups on Tesla K80
agg1	1.91	1.55
agg2	1.55	1.75

To demonstrate the effectiveness of our *sample-converge* technique, we performed experiments on two representative algorithms, which are agg1 and agg2. These two algorithms are user-defined aggregations collected from [20] and they both satify the conditions to converge sample points, which are described in Section 3.5. Table 4 shows the speedups when sample converge is enabled for parallel merge implementations of these two algorithms. In all cases, we use the execution times when they achieve the best performance. Clearly, *sample-converge* technique can substantially improve performance on both GPUs since it reduces the overhead of communication and computation. To demonstrate the effectiveness of data transformations, we performed experiments on an Nvidia Tesla P100 and an Nvidia Tesla K80. Figure 8 and 9 compare the speedups with and without data transformations for parallel merge implementation on these two GPUs, respectively. As indicated in the figures, the performance is several times better with data transformations n both GPUs because of coalesced memory accesses.

5.4 Scalability

In order to demonstrate the effectiveness of our parallel merge implementation method, we implemented sequential merge versions for the same set of algorithms. In sequential merge versions, we also adopted GPU related optimizations, such as data transformation, persistent-thread model, and others that were used for parallel merge. Figure 10 compares the speedups of sequential merge and

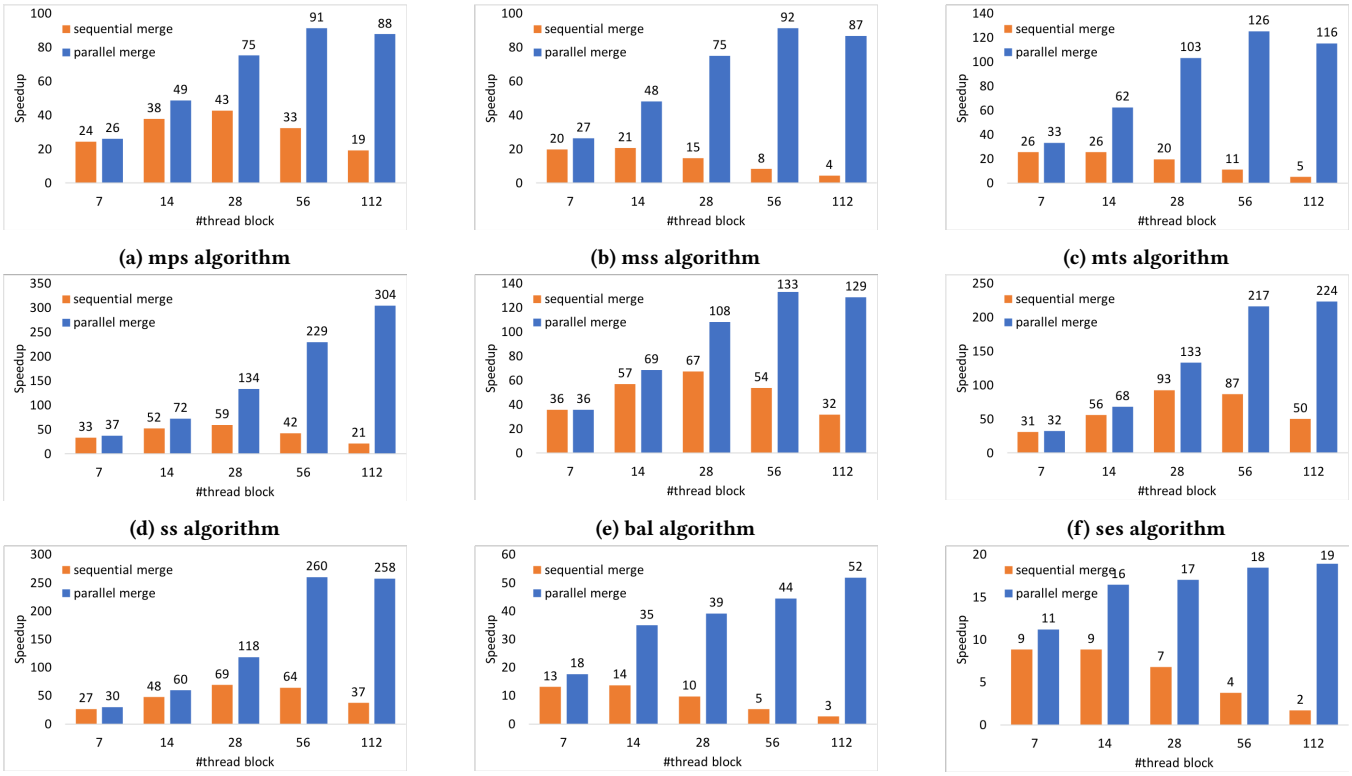


Figure 10: Comparison of sequential merge implementation and parallel merge implementations on different algorithms on an Nvidia Tesla P100

parallel merge on an Nvidia Tesla P100. The speedups are measured over sequential programs. As indicated in the figure, sequential merge and parallel merge deliver similar performance when the number of thread blocks is 7. This is because parallelism resources of GPUs are not fully utilized with such a small number of thread blocks. Thus, the merge stage is not a bottleneck at this stage. As the number of thread blocks continues to increase, speedups of both sequential merge and parallel merge grow. However, the performance gap between sequential and parallel merge also increases. The parallel merge versions are 4-20 times more efficient than the sequential merge at the end.

The experimental results show the difference of scalability of these two merge methods. The number of streaming processors on an Nvidia Tesla P100 is 56. For sequential merge programs, they achieve peak performance when the number of thread blocks is only 28 or even 14. When the number of thread blocks reaches 56, the speedup drops, which means that the computing resources are not fully utilized with the sequential merge. On the other hand, parallel merge programs show best performance with 56 thread blocks.

We also performed experiments on an Nvidia Tesla K80. Due to space limit, we briefly show the results in Table 5. The second and third column of the table show the maximal speedups over sequential programs achieved by sequential merge and parallel merge, respectively. As expected, parallel merge is 1.4-2.9 times more efficient than sequential merge. In addition, for parallel merge, the maximal speedups are achieved when the number of thread blocks is 26, which is equal to *maximal launch* in Table 2. However, with sequential merge, the number of thread blocks is smaller.

Table 5: Experimental results on an Nvidia Tesla K80. The numbers within parentheses are the number of thread blocks when they achieve the maximal speedups over sequential programs.

Algorithms	Speedups of sequential merge	Speedups of parallel merge
mps	14.51 (13)	24.25 (26)
mss	9.32 (13)	24.06 (26)
mts	14.54 (6)	29.58 (26)
ss	25.77 (13)	74.66 (26)
bal	25.07 (13)	35.93 (26)
ses	31.30 (13)	54.17 (26)
des	31.14 (13)	55.74 (26)
agg1	9.83 (13)	12.25 (26)
agg2	3.87(13)	5.62 (26)

6 RELATED WORK

Speculation Based Methods: A commonly used method to break dependences in loops is speculation[5, 18, 22]. For speculative execution, the actual values are checked at the runtime by the system.

If actual values do not match one or more of the speculated values, they simply rollback and re-execute the corresponding dataset. Zhao *et al.* [25, 26] parallelized a set of finite state machines (FSMs) with principled speculation. Qiu *et al.* [17] observed that speculative FSM parallelization is not scalable with an increasing number of cores. To address the limitation, they presented a series of scalability analysis models and developed an automatic speculative FSM parallelization framework named S3. For each run of FSM algorithm, S3 can automatically compute the optimal number of cores to use, so that it achieves best performance and reduces the waste of computing resources. Unlike this work, we propose to apply prefix sum parallelism pattern on a wider range of algorithms using a set of runtime check conditions to solve the scalability limitations.

Prefix Sum Parallelism on GPUs: There is a growing literature on improving the performance of parallel-prefix computations, especially on GPU architectures [2, 6, 14, 15, 24]. Some of the optimization techniques in this work, such as the persistent-thread model, are adopted in our implementation. Dotsenko *et al.*[6] proposed a matrix-based scan algorithm and extended it to segmented scan and backward scan algorithms. Breitbart [2] proposed a scan algorithm on GPUs that involved static workgroups to effectively allow manual work distribution with a fixed number of threads. Maleki *et al.* [15] proposed a massively-parallel prefix sum algorithm named SAM, which natively supports two generalized prefix sums, higher-order prefix sums and tuple-based prefix sums. They further generalize prefix sum implementations on GPUs to support linear recurrences [14]. Mytkowicz *et al.* [16] proposed to apply parallel prefix sum on FSM algorithms, which emulates all states. To reduce the redundancy, they adopt state convergence method, which has been made more broadly applicable in our work. Compared with these reasearch work, our work can benefit more classes of algorithms based on sampling-and-reconstruction method [12]. Thus, all of the above algorithms, including, prefix sum, higher-order prefix sums, tuple-based prefix sums, linear recurrences, and FSMs can be directly accelerated using our method.

7 CONCLUSION

With experimental evidence, this work points out that a principal limitation of sampling-and-reconstruction method is that it is not able to achieve linear scalability when being ported to many-core architectures such as GPUs. To overcome this fallency, this work enables prefix sum parallelism pattern with sampling-and-reconstruction method. This work also introduces a set of refined runtime check conditions, which can avoid almost all runtime check failures. In addition, we propose our methods to reduce reprocessing overhead and reduce workload through reducing the number of sample points. Evaluation results on a set of representative algorithms demonstrate that our methods deliver linear scalability and can be several times more efficient than sequential merge implementation.

REFERENCES

[1] [n. d.]. Parallel Prefix Sum. https://en.wikipedia.org/wiki/NC_(complexity).

[2] Jens Breitbart. 2010. Static GPU threads and an improved scan algorithm. In *European Conference on Parallel Processing*. Springer, 373–380.

[3] D. Callahan. 1992. Recognizing and parallelizing bounded recurrences. In *Languages and Compilers for Parallel Computing*, Utpal Banerjee, David Gelernter, Alex Nicolau, and David Padua (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 169–185.

[4] Ernie Chan, Marcel Heimlich, Avi Purkayastha, and Robert Van De Geijn. 2007. Collective communication: theory, practice, and experience. *Concurrency and Computation: Practice and Experience* 19, 13 (2007), 1749–1783.

[5] Chen Ding, Xipeng Shen, Kirk Kelsey, Chris Tice, Ruke Huang, and Chengliang Zhang. 2007. Software behavior oriented parallelization. In *ACM SIGPlan Notices*, Vol. 42. ACM, 223–234.

[6] Yuri Dotsenko, Naga K Govindaraju, Peter-Pike Sloan, Charles Boyd, and John Manferdelli. 2008. Fast scan algorithms on graphics processors. In *Proceedings of the 22nd annual international conference on Supercomputing*. ACM, 205–213.

[7] Azadeh Farzan and Victor Nicolet. 2017. Synthesis of Divide and Conquer Parallelism for Loops. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. ACM, New York, NY, USA, 540–555. https://doi.org/10.1145/3062341.3062355

[8] Grigory Fedyukovich, Maaz Bin Safeer Ahmad, and Rastislav Bodik. 2017. Gradual Synthesis for Static Parallelization of Single-pass Array-processing Programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. ACM, New York, NY, USA, 572–585. https://doi.org/10.1145/3062341.3062382

[9] Allan L. Fisher and Anwar M. Ghuloum. 1994. Parallelizing Complex Scans and Reductions. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI '94)*. ACM, New York, NY, USA, 135–146. https://doi.org/10.1145/178243.178255

[10] Kshitij Gupta, Jeff A Stuart, and John D Owens. 2012. A study of persistent threads style GPU programming for GPGPU workloads. In *Innovative Parallel Computing-Foundations & Applications of GPU, Manycore, and Heterogeneous Systems (INPAR 2012)*. IEEE, 1–14.

[11] Peng Jiang and Gagan Agrawal. 2017. Combining SIMD and Many/Multi-core parallelism for finite state machines with enumerative speculation. *ACM SIGPLAN Notices* 52, 8 (2017), 179–191.

[12] Peng Jiang, Linchuan Chen, and Gagan Agrawal. 2018. Revealing parallel scans and reductions in recurrences through function reconstruction. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*. ACM, 10.

[13] Peter M. Kogge and Harold S. Stone. 1973. A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations. *IEEE Trans. Comput.* 22, 8 (Aug. 1973), 786–793. https://doi.org/10.1109/TC.1973.5009159

[14] Sepideh Maleki and Martin Burtscher. 2018. Automatic Hierarchical Parallelization of Linear Recurrences. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 128–138.

[15] Sepideh Maleki, Annie Yang, and Martin Burtscher. 2016. *Higher-order and tuple-based massively-parallel prefix sums*. Vol. 51. ACM.

[16] Todd Mytkowicz, Madanlal Musuvathi, and Wolfram Schulte. 2014. Data-parallel finite-state machines. In *ACM SIGARCH Computer Architecture News*, Vol. 42. ACM, 529–542.

[17] Junqiao Qiu, Zhijia Zhao, Bo Wu, Abhinav Vishnu, and Shuaiwen Leon Song. 2017. Enabling scalability-sensitive speculative parallelization for fsm computations. In *Proceedings of the International Conference on Supercomputing*. ACM, 2.

[18] Easwaran Raman, Ram Rangan, David I August, et al. 2008. Spice: speculative parallel iteration chunk execution. In *Proceedings of the 6th annual IEEE/ACM international symposium on Code generation and optimization*. ACM, 175–184.

[19] Veselin Raychev, Madanlal Musuvathi, and Todd Mytkowicz. 2015. Parallelizing User-defined Aggregations Using Symbolic Execution. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*. ACM, New York, NY, USA, 153–167. https://doi.org/10.1145/2815400.2815418

[20] Veselin Raychev, Madanlal Musuvathi, and Todd Mytkowicz. 2015. Parallelizing user-defined aggregations using symbolic execution. In *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, 153–167.

[21] Shigeyuki Sato and Hideya Iwasaki. 2011. Automatic Parallelization via Matrix Multiplication. In *Proceedings of the 32Nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)*. ACM, New York, NY, USA, 470–479. https://doi.org/10.1145/1993498.1993554

[22] Chen Tian, Min Feng, and Rajiv Gupta. 2010. Speculative parallelization using state separation and multiple value prediction. *ACM Sigplan Notices* 45, 8 (2010), 63–72.

[23] Shucai Xiao and Wu-chun Feng. 2010. Inter-block GPU communication via fast barrier synchronization. In *Parallel & Distributed Processing (IPDPS), 2010 IEEE International Symposium on*. IEEE, 1–12.

[24] Shengen Yan, Guoping Long, and Yunquan Zhang. 2013. StreamScan: fast scan algorithms for GPUs without global barrier synchronization. In *ACM SIGPLAN Notices*, Vol. 48. ACM, 229–238.

[25] Zhijia Zhao and Xipeng Shen. 2015. On-the-fly principled speculation for FSM parallelization. In *ACM SIGPLAN Notices*, Vol. 50. ACM, 619–630.

[26] Zhijia Zhao, Bo Wu, and Xipeng Shen. 2014. Challenging the embarrassingly sequential: parallelizing finite state machine-based computations through principled speculation. In *ACM SIGARCH Computer Architecture News*, Vol. 42. ACM,

543–558.